

Guía para ejecutar las pruebas de validación

Plataforma web para la conversión de audio estéreo multifuente a formato Ambisónico

Propósito. Explicar qué sistema debe usarse en cada prueba: notebook original, plataforma web o script independiente de análisis. También define los archivos de entrada, las métricas, las evidencias y la referencia utilizada para interpretar cada resultado.

1. Tres elementos distintos que participan en la validación

Elemento	Para qué se usa	No debe usarse para
Notebook original	Generar la salida sonora de referencia con la lógica original del proyecto.	Medir el rendimiento real de la plataforma integrada.
Página local / backend	Generar la salida del sistema que se está validando y medir el tiempo total de uso.	Ser considerada automáticamente idéntica al notebook sin compararla.
Scripts de validación	Leer las salidas, calcular RMS, correlación, espectro, clipping, tiempos y crear gráficas.	Modificar el audio o reemplazar el algoritmo del notebook y de la página.

2. Aclaraciones sobre el procesamiento

Funciones Python directas

Significa que FastAPI llama funciones importadas desde `processor.py` y `core_dsp.py`. Ya no abre ni ejecuta un archivo .ipynb mediante Papermill. El cálculo DSP sigue existiendo; lo que se elimina es la sobrecarga de iniciar y ejecutar un notebook completo en cada solicitud.

Procesamiento por bloques

No significa que todos los bloques se procesen al mismo tiempo. Normalmente se leen y procesan de manera secuencial: bloque 1, bloque 2, bloque 3, etc. La ventaja es que no es necesario cargar todo el audio largo en RAM. Las colas de convolución y el estado espacial deben conservarse entre bloques.

STFT e ISTFT

La conversión al dominio de la frecuencia y el regreso al dominio temporal todavía se realizan cuando el algoritmo las necesita. No dejaron de consumir tiempo; simplemente se evitó repetir operaciones innecesarias y se reorganizó la ejecución. La optimización no elimina la matemática del algoritmo.

SOFA y HRTF

SOFA no es una colección de miles de audios completos. Es un contenedor estandarizado que guarda respuestas impulsionales HRTF medidas para muchas direcciones, junto con sus coordenadas y metadatos. Cada HRTF es un filtro corto para el oído izquierdo y derecho.

Convolución

Es la operación que aplica un filtro HRTF a una señal. En términos prácticos, combina la señal de una dirección virtual con las respuestas impulsionales izquierda y derecha para producir el audio binaural. Las variables concretas dependen del código, pero suelen ser la señal virtual y los filtros IR/HRTF de cada oído. La documentación del proyecto debe identificar sus nombres exactos.

Dónde se usa la HRTF

La HRTF se utiliza en la etapa de render binaural, es decir, para convertir la representación espacial FOA en dos canales destinados a audífonos. No es la operación que crea inicialmente el FOA desde el estéreo; esa conversión utiliza las funciones de codificación espacial del DSP.

3. Banco de archivos recomendado

Código	Archivo	Duración	Uso principal
A01	impulso.wav	1-2 s	Estructura FOA, HRTF, ITD/ILD y clicks
A02	tono_1kHz.wav	10-30 s	RMS, clipping, espectro y ganancia
A03	ruido_blanco.wav	30 s	Respuesta espectral y estabilidad
A04	musica_corta.wav	30 s	Comparación notebook vs página
A05	musica_1min.wav	1 min	Calidad y rendimiento
A06	musica_5min.wav	5 min	Streaming, RAM y rendimiento
A07	musica_10min.wav	10 min	Criterio máximo de rendimiento

Protocolo 1. Verificación estructural del FOA

Dónde se ejecuta: Script de validación que llame directamente las funciones DSP de core_dsp.py. No basta con escuchar el MP3 de la página.

Archivos: A01, A02 y señales direccionales controladas para frente, atrás, izquierda, derecha, arriba y abajo.

Procedimiento

1. Crear o usar señales mono controladas.
2. Codificarlas con las funciones FOA reales del proyecto.
3. Guardar temporalmente W, X, Y y Z como WAV de cuatro canales o como arreglo NumPy.
4. Verificar canales, orden, frecuencia, duración, NaN/Inf, pico y energía por canal.
5. Comprobar el signo y predominio esperado de X, Y y Z para cada dirección.

Métricas: Número de canales, orden, frecuencia, muestras, RMS por canal, signo dominante, NaN/Inf y clipping.

Referencia y criterio: Convención FOA declarada por el proyecto (por ejemplo, AmbiX/ACN/SN3D) y posición programada.

Evidencias: Tabla por dirección, gráfica de RMS de W/X/Y/Z y fragmento del código que define el orden.

Prueba o parámetro	Resultado	Valor esperado	Cumple	Observaciones

Protocolo 2. Fidelidad: notebook original frente a página

Dónde se ejecuta: Notebook original para producir la referencia; página local para producir la salida nueva; script independiente para comparar.

Archivos: A02, A03, A04 y A05. Usar exactamente el mismo archivo y los mismos parámetros.

Procedimiento

1. Procesar el archivo con el notebook original y guardar la salida WAV de referencia.
2. Procesar el mismo archivo desde la página y descargar la salida WAV equivalente.
3. No comparar MP3 si se busca equivalencia numérica, porque la compresión añade diferencias.
4. Alinear ambas señales en muestras y, si es necesario, corregir solo un retraso global documentado.
5. Calcular correlación, error RMS, error máximo, diferencia de pico, loudness y espectro.
6. Escuchar la señal diferencia: referencia menos página.

Métricas: Correlación, RMSE, error máximo, pico, RMS, loudness, diferencia espectral, clipping, NaN/Inf.

Referencia y criterio: Salida del notebook original.

Evidencias: Tabla de métricas, espectros superpuestos, forma de onda de la diferencia y audios utilizados.

Prueba o parámetro	Resultado	Valor esperado	Cumple	Observaciones

Protocolo 3. Continuidad del procesamiento por bloques

Dónde se ejecuta: Página/backend en el modo utilizado para audios largos; script independiente para analizar fronteras.

Archivos: A06 o A07, preferiblemente WAV, con contenido continuo.

Procedimiento

1. Procesar el audio largo en la página.
2. Guardar la salida WAV.
3. Identificar el tamaño real de bloque utilizado por processor.py.
4. Analizar ventanas de al menos 100 ms antes y después de cada frontera.
5. Graficar forma de onda, RMS por ventanas y diferencia de muestras.
6. Escuchar específicamente alrededor de cada frontera.

Métricas: Saltos de amplitud, cambio de RMS, clicks, discontinuidades, clipping y estabilidad de loudness.

Referencia y criterio: Continuidad de la señal y, cuando exista, salida en memoria/notebook del mismo audio.

Evidencias: Gráficas alrededor de cada frontera y tabla con instante, salto máximo y observación auditiva.

Prueba o parámetro	Resultado	Valor esperado	Cumple	Observaciones

Protocolo 4. Rendimiento de la plataforma

Dónde se ejecuta: Página local o desplegada, porque se evalúa la experiencia completa. El backend puede medirse adicionalmente para diagnóstico.

Archivos: A04, A05, A06 y A07; WAV y MP3; 44.1 y 48 kHz si la plataforma los admite.

Procedimiento

1. Hacer una ejecución de calentamiento.
2. Procesar 10 s, 30 s y 1 min cinco veces; 5 y 10 min tres veces.
3. Registrar desde pulsar Procesar hasta que aparecen los enlaces.
4. Registrar también processingTime, duración, RAM y CPU si están disponibles.
5. Calcular porcentaje = tiempo total / duración x 100.
6. Reportar promedio, mínimo, máximo y desviación estándar.

Métricas: Tiempo total, tiempo backend/DSP, porcentaje, RAM máxima, CPU y tasa de éxito.

Referencia y criterio: Requisito del proyecto: tiempo total menor o igual al 60 % de la duración, si ese criterio queda aprobado por el director.

Evidencias: Tabla por duración y formato, gráfica duración vs tiempo y especificaciones del computador.

Prueba o parámetro	Resultado	Valor esperado	Cumple	Observaciones

Protocolo 5. Comparación con software ambisónico de referencia

Dónde se ejecuta: Plataforma, herramienta seleccionada (por ejemplo IEM Plug-in Suite) y script independiente.

Archivos: A01, A02 y una señal musical mono corta.

Procedimiento

1. Configurar igual frecuencia, dirección, orden y normalización.
2. Generar FOA en la plataforma o mediante su función DSP.
3. Generar FOA en el software de referencia.
4. Convertir el orden/normalización si no coinciden.
5. Alinear y comparar canal por canal.
6. Decodificar ambas salidas con el mismo render binaural cuando sea posible.

Métricas: Correlación por canal, RMS, pico, espectro, señal diferencia y prueba auditiva AB/ABX.

Referencia y criterio: Software ambisónico de referencia con configuración equivalente.

Evidencias: Capturas de configuración, tabla por canal, gráficas y archivos exportados.

Prueba o parámetro	Resultado	Valor esperado	Cumple	Observaciones

Protocolo 6. Evaluación perceptual y usabilidad

Dónde se ejecuta: Página y audios previamente preparados; formulario de respuestas.

Archivos: Audios binaurales de posiciones conocidas y tareas de carga, conversión y descarga.

Procedimiento

1. Definir posiciones objetivo y ocultarlas al participante.
2. Usar audífonos y volumen constante.
3. Solicitar dirección percibida y calificación de calidad.
4. Pedir al participante completar tareas en la página.
5. Registrar errores, tiempo, dudas y éxito.
6. Calcular error angular, aciertos y puntuación de usabilidad.

Métricas: Error angular, aciertos por cuadrante, frente/atrás, valoración 1-5, tiempo de tarea, errores y SUS si se aplica.

Referencia y criterio: Posición programada y criterios de usabilidad definidos antes de la prueba.

Evidencias: Consentimiento/registro, hoja de respuestas, tabla agregada y gráficas.

Prueba o parámetro	Resultado	Valor esperado	Cumple	Observaciones

Protocolo 7. Base de datos y funcionamiento integrado

Dónde se ejecuta: Página, API, consultas a la base de datos y sistema de archivos.

Archivos: Archivos válidos, un archivo no compatible y varias sesiones de visitante.

Procedimiento

1. Iniciar una sesión y comprobar visitor_sessions.
2. Enviar una sugerencia y comprobar su relación con la sesión.
3. Subir y procesar un audio.
4. Comprobar metadatos, estado y rutas en conversion_jobs/audio_files si esas tablas están integradas en la versión final.
5. Verificar que la ruta almacenada apunta a un archivo real.
6. Forzar un error controlado y verificar el estado failed.
7. Confirmar que la base guarda metadatos y rutas, no el contenido binario del audio.

Métricas: Pruebas superadas/ejecutadas, consistencia de campos, estados válidos, rutas existentes y errores controlados.

Referencia y criterio: Esquema real de la base de datos, contratos de la API y comportamiento funcional definido.

Evidencias: Consultas SQL, respuestas JSON, capturas y tabla pass/fail.

Prueba o parámetro	Resultado	Valor esperado	Cumple	Observaciones

4. Scripts independientes que conviene crear

Script sugerido	Función
validation/analyze_foa_structure.py	Genera y revisa W/X/Y/Z con señales controladas.
validation/compare_notebook_vs_web.py	Alinea dos WAV y calcula correlación, RMSE, pico y señal diferencia.
validation/analyze_audio_quality.py	Calcula RMS, clipping, NaN/Inf, DC offset, crest factor y espectro.
validation/analyze_block_boundaries.py	Analiza clicks y cambios de nivel alrededor de cada frontera.
validation/benchmark_platform.py	Organiza tiempos, porcentajes, repeticiones y estadísticas.
validation/analyze_itd_ild.py	Calcula diferencias interaurales en impulsos binaurales.

Regla metodológica: no agregar las métricas al notebook o al backend solo para poder evaluarlos. Los scripts de validación deben leer los archivos resultantes sin alterar el sistema evaluado.

5. Resumen rápido

Pregunta	Respuesta
¿Con qué se verifica fidelidad?	Notebook original vs salida WAV de la página.
¿Con qué se mide rendimiento?	Con la página completa y, adicionalmente, con logs del backend.
¿Con qué se hacen RMS y gráficas?	Con scripts Python independientes.
¿Todos los bloques se ejecutan juntos?	No necesariamente; normalmente se procesan en secuencia.
¿SOFA contiene canciones?	No; contiene filtros/respuestas impulsionales HRTF y metadatos espaciales.
¿La HRTF crea el FOA?	No; se usa principalmente para renderizar FOA a binaural.
¿STFT/ISTFT ya no se usan?	Sí se usan si el algoritmo las requiere; la optimización evita sobrecarga y repeticiones.